

A CONCRETE EXTENSION TO THE FORGE DESIGN

Certified Finite Summaries for Infinite Proofs

Weighted words, boundary-safe telescoping,
and singular recurrence transport

Prepared for Vladimir Reshetnikov

15 September 2026

Abstract

Forge already proposes a broad certificate-producing planner around Lean’s `grind`. This article adds three concrete mathematical capabilities rather than another planner architecture: exact equivalence checking for finite-dimensional word computations, creative telescoping with explicitly safe support boundaries, and recurrence proof composition that accounts for singular leading coefficients. A finite matrix simulation can prove an identity for every list, including lists over an infinite polynomial input alphabet. A shifted-binomial flux can prove a recurrence for a definite hypergeometric sum without treating an undefined product at a boundary as zero. Checked left multiples of Ore operators then let independently obtained recurrences meet, while a finite singularity cover determines which additional values must be proved equal.

The accompanying Python implementation produces 245 checkable entries in one recorded corpus: 54 matrix closure certificates, 115 separating words, 23 summation certificates, six Ore identities, and 47 singularity plans. All replay with Python site packages disabled. The corpus includes 100 differential word-model tests, 245 rejected invalidating mutations, and two honestly recorded insufficient-ansatz outcomes; 41 additional unit tests pass. These are exact Python checks, not Lean proofs. The article gives soundness arguments, precise fragment boundaries, a library map, proposed Lean modules, and implementation acceptance gates. No integrated tactic, kernel-checked certificate, or performance comparison against an existing Lean tactic is claimed.

Audited Forge baseline: 674521027d968d59f7b83220ed52304a85cb55e2.

Companion bundle: source, executable prototypes, raw certificates, tests, and run history.

Contents

1	The increment: new finite proof objects, not a second planner	3
1.1	What the repository already supplies	3
1.2	A precise non-duplication claim	3
1.3	What an actual gain would look like	4
1.4	Evidence levels used in this article	4
2	Worker I: finite-dimensional proofs for all words	4
2.1	Model and certificate	4
2.2	Search: exact reachable-space closure	5
2.3	Complexity, proof size, and useful nonminimal certificates	6
2.4	Infinite input alphabets by polynomial interpolation	6
2.5	Affine state updates and polynomial observations	7
2.6	What the front end must reject or leave as an obligation	8
3	Worker II: creative telescoping without boundary fiction	8
3.1	A different problem from polynomial antidifferencing	8
3.2	Why a rational multiplier is not enough	9
3.3	The shifted-support ansatz	9
3.4	Interior verification by a polynomial identity	10
3.5	Every exceptional boundary is a separate checked identity	10
3.6	How the search is implemented	11
3.7	Recovered certificates, not just guessed recurrences	12
3.8	Lean-specific traps and the initial support library	13
4	Worker III: recurrence transport with singularity-aware seeds	13
4.1	Why two good recurrence proofs may not meet	13
4.2	Noncommutative multiplication is part of the certificate	14
4.3	A concrete common multiple	14
4.4	The singularity problem	15
4.5	A selective seed theorem	15
4.6	A complete finite cover of natural singularities	16
4.7	Apparent singularities introduced by transport	16
4.8	Connecting all three workers	17
5	Concrete Lean integration, with no new global controller	17
5.1	Reuse Forge’s boundary; add worker-specific semantics	17
5.2	The smallest useful word-fold reifier	17
5.3	The matrix soundness theorem is smaller than the search	18
5.4	Summation preparation and residual obligations	19
5.5	Proposed modules and concrete responsibilities	19

5.6	Library map: use existing mathematics and search engines	20
6	A specific Leant/Djex extension: quantified equivalence of eligible folds	20
6.1	The gap to target	20
6.2	An operational candidate pipeline	21
6.3	Semantic deduplication that does not discard evidence	21
6.4	Counterexamples improve search without becoming oracles of truth	21
7	Executed evidence and what it measures	22
7.1	The authoritative run	22
7.2	Composition of the word corpus	22
7.3	Summation and recurrence evidence	23
7.4	Invalidating changes and independent unit tests	23
7.5	Failed runs and repairs are retained	24
7.6	What these experiments cannot establish	24
8	Implementation priorities and acceptance gates	25
8.1	First increment: one real matrix proof path	25
8.2	Second increment: the signed-support telescoping theorem	25
8.3	Third increment: operator action and selective seeds	25
8.4	Fourth increment: the Leant consumer	25
8.5	A fair future tactic evaluation	26
9	Conclusion	26
A	Artifact protocol and reproduction	27
A.1	Files and commands	27
A.2	Canonical rational and polynomial encoding	27
A.3	Authoritative problem versus certificate	28
A.4	Checker map	28
B	Scope ledger for future claims	28

1 The increment: new finite proof objects, not a second planner

1.1 What the repository already supplies

At the audited revision, Forge is a design and research-prototype repository, not an installed tactic. Its merged article already covers induction with accumulator generalisation, polynomial recurrence and invariant synthesis, several arithmetic witness engines, nonlinear certificates, demand-directed Horn reasoning, proof-logging Boolean search, and the separation between search and checked evidence. Its integration section already advocates a conservative bridge through ordinary Lean goals and explicitly proved facts. Those ideas are prerequisites here, not contributions of this article [1, 2, 3, 4].

The useful next question is not “which extra tactic should a scheduler try?” It is: *which large families of proof obligations admit a finite object whose verification is much simpler than discovering the proof?* This question leads to three workers with sharp input and output contracts.

Worker	Input family	Returned object	New scope relative to the audited design
Linear words	A finite-dimensional rational state model and an output identity	An invariant row-space simulation, or a separating word	All words, not just an additive polynomial sequence or one proposed invariant
Safe telescoping	A definite binomial-power sum	A recurrence plus a shifted-support flux	Hypergeometric summands and moving support, not polynomial antidifferencing
Recurrence transport	Two annihilating operators	A common left multiple and singularity/seed plan	Composition of nonconstant recurrences without silently cancelling singular leading terms

Table 1: The three mathematical additions. A singularity plan still leaves explicit equality obligations.

The workers also connect. A unary linear word model yields a constant-coefficient recurrence. A telescoper yields a polynomial-coefficient recurrence. A common left multiple gives both objects one recurrence, and singularity-aware induction reduces equality to finitely many proved values. None of these connections requires replacing `grind`’s equality or arithmetic engines.

1.2 A precise non-duplication claim

The comparison was made against Forge’s README, implementation-status ledger, Leant/Djex transfer guide, structural-search and integration sections, conclusion, and provenance appendix at commit 674521027d968d59f7b83220ed52304a85cb55e2. The provenance appendix maps the merged proposals to their nine original submissions [5]. This is not a claim to have read every line of all nine frozen originals. Repository code search returned no matches even for a term visibly present in the sources, so negative search results were not used to establish absence. The companion docs/NON_DUPLICATION.md records this limitation.

The underlying mathematics is not presented as new. Field-weighted automaton equivalence and invariant-space algorithms are established methods; Kiefer’s notes give a particularly direct account [9]. Creative telescoping and Ore-operator computation likewise have mature implementations [12, 10]. The contribution is the concrete selection of fragments, the proof-object

interfaces, the boundary-aware implementation, and their proposed use inside Forge. An existing polynomial-invariant engine could prove some overlapping examples after a suitable invariant was supplied. What changes here is the automatic reduction to a finite closure calculation, with a completeness theorem for an explicitly stated fragment.

1.3 What an actual gain would look like

Consider a theorem about two different arithmetic folds over an arbitrary list. A local saturation engine may simplify their definitions, but simplification does not itself invent the relation connecting their hidden states. The linear worker computes an invariant space in which that relation becomes a single orthogonality check. Its proof is independent of list length.

Now consider a sum of cubes of binomial coefficients. A polynomial template in n is inappropriate: the sequence is not being claimed polynomial. The summation worker instead proposes a recurrence and a flux whose finite difference is the desired combination of summands. The sum disappears only after both support boundaries have been justified.

Finally, two valid recurrence proofs need not use the same operator. Their common left multiple provides a shared induction rule. If that operator has a zero leading coefficient at a natural index, the proof must request an extra value rather than pretend that its usual recurrence step still determines the next term. This is a mathematical obligation, not a larger search budget.

1.4 Evidence levels used in this article

Evidence status. All Python workers and tests described as executed were run. The Lean source supplied in the bundle is an uncompiled semantic-lemma candidate. Lean, Lake, and Elan were not available in the working runtime; attempted retrieval routes did not establish an executable toolchain.

A mathematical proof below establishes the stated theorem under its hypotheses. A passing Python checker establishes that this implementation accepted a particular encoded object. Neither alone establishes that an arbitrary Lean expression was faithfully encoded, or that a Lean kernel accepted the resulting proof. The proposed Lean acceptance path must discharge all of those separate requirements. The raw ledger never upgrades “plan accepted” into “theorem proved”.

Forge’s own conclusion correctly distinguishes additional prototype results from closing its Lean-integration gap [6]. This article does not erase that gap. Its first implementation milestone is correspondingly a small matrix soundness theorem and source bridge, not three simultaneous large integrations.

2 Worker I: finite-dimensional proofs for all words

2.1 Model and certificate

Let A be a finite alphabet and $d \geq 1$. A rational linear word model consists of an initial row $u \in \mathbb{Q}^{1 \times d}$, a matrix $M_a \in \mathbb{Q}^{d \times d}$ for every $a \in A$, and an output column $v \in \mathbb{Q}^{d \times 1}$. For $w = a_1 \cdots a_m$, put

$$M_w = M_{a_1} \cdots M_{a_m}, \quad M_\epsilon = I, \quad f(w) = uM_w v.$$

The worker decides whether $f(w) = 0$ for every finite word. Equality of two models reduces to this problem by block-diagonal composition:

$$u = (u_1, u_2), \quad M_a = \text{diag}(M_{1,a}, M_{2,a}), \quad v = \begin{pmatrix} v_1 \\ -v_2 \end{pmatrix}.$$

This reduction requires the same alphabet and an exact correspondence between its symbols. It does not identify two source constructors because they have similar printed names.

A positive certificate contains an arbitrary matrix $B \in \mathbb{Q}^{b \times d}$, a row $\alpha \in \mathbb{Q}^{1 \times b}$, and matrices $C_a \in \mathbb{Q}^{b \times b}$ satisfying

$$u = \alpha B, \quad BM_a = C_a B \quad (a \in A), \quad Bv = 0. \quad (1)$$

The search usually chooses independent rows for B , but the checker does *not* need to prove independence, minimality, or even reachability of those rows. Any finite simulation satisfying these identities suffices. The rank-zero case is legitimate: it certifies exactly the zero initial row.

Theorem 2.1 (Soundness of the word certificate). *If (1) holds, then $uM_wv = 0$ for every $w \in A^*$.*

Proof. Induction on the word gives $BM_w = C_w B$, where C_w is the analogous product of the small matrices. Consequently $uM_wv = \alpha BM_wv = \alpha C_w Bv = 0$. The empty word uses the identity matrices. Every equality is a matrix identity over the same rational field. $\square$

The certificate says more than “these examples worked”. It provides a finite state representation that is closed under every transition and whose entire represented space has output zero. This is the missing universal argument.

2.2 Search: exact reachable-space closure

Define the reachable row space

$$\mathcal{R} = \text{span}_{\mathbb{Q}}\{uM_w : w \in A^*\}.$$

A breadth-first agenda stores a row basis for this space together with a word that generates each stored row. The procedure is elementary.

```

check u * v; return the empty word if it is nonzero
insert u if independent; queue its basis index
while the queue is nonempty:
  take one stored row b with generating word w
  for each letter a:
    y := b * M[a]
    if y * v != 0: return the concrete word w ++ [a]
    if y is independent of the stored rows:
      insert y and queue it, retaining w ++ [a]
solve u = alpha * B and B * M[a] = C[a] * B
return B, alpha, C

```

The implementation incrementally maintains echelon rows and their exact coordinates in the original, word-labelled rows. It never uses a floating-point rank tolerance. A word is checked against the original model, not against the normalised echelon rows, which need not themselves be reachable by a single word.

Theorem 2.2 (Termination and completeness of the rational fragment). *Without a resource interruption, reachable-space search terminates and returns either a valid certificate (1) or a word with nonzero output. A separating word exists of length at most $d - 1$ whenever the universal identity is false.*

Proof. Every admitted row is independent of the previous rows, so at most d rows are admitted. Each is expanded once under each letter. At agenda exhaustion, the stored span contains u and

is closed under all M_a , so it contains every reachable row. Conversely, every admitted row is reachable. Thus it equals $\mathcal{R}$, and the coordinate equations for the certificate are solvable. Every stored or tested row is checked against v ; if none separates, the certificate satisfies $Bv = 0$.

For the length bound, let R_j be the span of reachable rows from words of length at most j . If $R_j = R_{j+1}$, it is already transition-closed and all later spaces agree. Starting with dimension at most one, the ascending chain must either stabilise earlier or reach its maximum dimension by $j = d - 1$. Therefore $R_{d-1} = \mathcal{R}$. A nonzero linear functional on $\mathcal{R}$ is nonzero on at least one of the word generators of R_{d-1} . $\square$

This is the standard weighted-automaton argument, specialised to a small certificate interface [9]. The new checker deliberately does less than the search: it multiplies rational matrices and compares the results. It need not trust how Gaussian elimination was implemented.

2.3 Complexity, proof size, and useful nonminimal certificates

Incremental reduction and dense matrix-vector multiplication take $O(d^2)$ field operations per candidate row. There are at most $|A|d$ expansions, so the basic search is $O(|A|d^3)$ field operations. Dense verification of (1) costs

$$O(|A|(bd^2 + b^2d) + bd), \quad b \leq d$$

for certificates emitted by this search. Storage is $O(bd + |A|b^2)$ rational entries, in addition to the original model. These are *field-operation* bounds, not constant-time bit bounds. Intermediate rational numerators and denominators can grow substantially.

In a production search, fraction-free elimination or modular discovery with exact reconstruction is worth considering. In the acceptance path, however, reconstruction must be followed by the same exact identities. A modular rank calculation is not evidence that a rational equality holds. The proposed FLINT backend is only a faster producer of candidates.

Minimising B is optional. A small nonminimal certificate can be much easier to find than an absolutely minimal one, and its algebraic checks remain sound. For example, the Cassini lift below has dimension ten and an emitted reachable basis of rank eight; no claim of output-minimal automaton construction is made. A later certificate optimiser can reduce rows if it provides another valid simulation. Proof size, not minimality terminology, is the practical objective.

2.4 Infinite input alphabets by polynomial interpolation

Many relevant folds consume arbitrary integers or rationals, not letters from a small enumerated datatype. Suppose the transition matrix depends polynomially on an input x :

$$M(x) = \sum_{j=0}^D x^j M_j, \quad M_j \in \mathbb{Q}^{d \times d}. \quad (2)$$

A positive certificate checks $BM_j = C_j B$ for every coefficient matrix. Then

$$BM(x) = \left(\sum_{j=0}^D x^j C_j \right) B$$

for every rational x , so Theorem 2.1 proves the identity for every finite sequence of rational inputs. The same positive algebraic argument transports to a compatible characteristic-zero field through its rational embedding; the source adapter must prove that transport.

There is also a complete finite search alphabet.

Theorem 2.3 (Interpolation alphabet). *For (2), choose $D + 1$ distinct rationals $t_0, \dots, t_D$. The reachable space generated by all $M(x)$, $x \in \mathbb{Q}$, equals the reachable space generated by $M(t_0), \dots, M(t_D)$.*

Proof. Every grid transition is an allowed rational transition, giving one inclusion. The Vandermonde matrix $(t_i^j)_{0 \leq i, j \leq D}$ is invertible, so each coefficient matrix M_j is a rational linear combination of the grid matrices. A row space closed under every grid matrix is consequently closed under every M_j and hence every $M(x)$. This gives the other inclusion. $\square$

The implementation uses the grid $0, 1, \dots, D$. It searches with those actual transitions, then emits closure actions for the *coefficient* matrices. A negative result is therefore a concrete list of grid inputs. This detail matters: directly searching the coefficient matrices could produce a sequence of coefficient indices that is not a legal sequence of source inputs.

The grid is not an empirical sample. Completeness rests on the proved degree bound. If that bound is understated, the conclusion is false. The test

$$s' = s + x(x - 1), \quad s_0 = 0$$

is zero for every word over inputs 0 and 1, but one input 2 yields output 2. The compiler computes degree two, and the worker returns [2] as a separating input. This is a regression control against treating a small input set as universal without checking interpolation hypotheses. For several input coordinates, a rectangular interpolation grid is a valid extension, but its size is a product of degree bounds. A sparse multivariate unisolvent set could improve search. Neither extension is implemented here.

2.5 Affine state updates and polynomial observations

A useful front end need not ask users to write matrices. Let the state be $z = (z_1, \dots, z_m)$, let each update be affine in z , and let the desired output relation be a polynomial $H(z)$ of degree at most e . Form the feature row

$$\Phi_e(z) = (z^\alpha)_{|\alpha| \leq e}, \quad \dim \Phi_e = \binom{m+e}{e}.$$

Substitution of an affine update into a polynomial of degree at most e does not increase its state degree. Thus each update T_a has a matrix satisfying

$$\Phi_e(T_a(z)) = \Phi_e(z)M_a, \quad H(z) = \Phi_e(z)v. \quad (3)$$

The initial feature row is $u = \Phi_e(z_0)$. Equations (3), followed by the matrix certificate, prove H vanishes on every reachable state.

Coefficients of an affine state update may themselves be polynomial in an input x . Substitution then gives matrices of form (2). The relevant input degree is the degree *after lifting*; it can exceed the degree in the unlifted update. The supplied compiler expands substitutions, extracts state monomial coefficients, determines input degree, and checks the symbolic substitution identities with SymPy. Its positive certificates are checked in the matrix model. This compiler is not yet an independently proved Lean reifier.

Example 2.4 (Two related Horner folds). Start at $(h, j, g) = (0, 0, 0)$ and consume a list through

$$h' = bh + x, \quad j' = bj + 2x + 3, \quad g' = bg + 1,$$

where b is a fixed rational. The desired identity is $j = 2h + 3g$ after every list. The degree-one features $(1, g, j, h)$ give a four-dimensional model with input degree one. The tested instances

$b = -2, 0, 1, 3$ each emit a rank-three closure certificate after six basis-letter expansions. A direct handwritten induction is short; the important automatic step is deriving and certifying the hidden state relation without being given that induction proof.

Example 2.5 (A polynomial observer, not a polynomial sequence). Start with $(a, b, c) = (0, 1, 1)$ and allow either transition

$$T_1(a, b, c) = (b, a + b, -c), \quad T_2(a, b, c) = (a + b, a + 2b, c).$$

The observer is $H = b^2 - ab - a^2 - c$. Since $T_2 = T_1^2$, words describe arbitrary blocks of one or two Fibonacci advances, and the identity is the Cassini relation with the parity sign carried in c . Degree-two lifting has ten features. Exact search emits a rank-eight certificate after sixteen expansions. This proves an output identity for an infinite collection of transition words; it does not claim that the Fibonacci values themselves are polynomials in the word length.

2.6 What the front end must reject or leave as an obligation

The first source fragment should be deliberately narrow: fixed rational coefficients, affine state updates, polynomial observations, and either an explicit finite transition alphabet or one polynomial input coordinate. A state update such as $z' = z^2$ is rejected by the supplied compiler. A broader Koopman-style lift may exist for some nonlinear systems, but a finite closure is not automatic and must not be presumed from a finite set of experiments.

Natural subtraction, integer division, overflow arithmetic, and tests on state values do not become rational affine operations merely because their notation resembles them. Casts require homomorphism proofs; a branch requires an exact case split or a documented over-approximation. If all branch actions are allowed in arbitrary order, positive closure is sound for the resulting larger model, but a negative word may be infeasible in the source program. Such a word is only guidance until source execution or a proved feasibility argument confirms it.

An unknown or unsupported translation is not a disproof. For a translated exact model, by contrast, a separating word is genuine positive evidence of an inequality. The certificate interface distinguishes these cases instead of routing both through a single “failed” result.

3 Worker II: creative telescoping without boundary fiction

3.1 A different problem from polynomial antidifferencing

Forge already searches for polynomial q satisfying $q(n+1) - q(n) = f(n)$ for a polynomial increment f . This is useful, but it is not a method for sums whose summands contain factorial or binomial quotients. The present worker targets

$$S_{p,q}(n) = \sum_{k=0}^n q^k \binom{n}{k}^p, \quad n \in \mathbb{N}, \quad p \in \{1, 2, \dots\}, \quad q \in \mathbb{Q}. \quad (4)$$

The concrete implementation bounds $p \leq 8$, recurrence order at most six, and polynomial degrees through a specified search request. Its experimental corpus uses $p \leq 4$. There is no claim that every possible request inside those decoder bounds will succeed.

Creative telescoping seeks a recurrence for S by proving a local identity for the summand. It is a mature method rather than a new proposed theory. The production recommendation is to reuse `HolonomicFunctions` for broader multivariate telescoping and `ore_algebra` for operator computations, while retaining a small independently checked certificate language [12, 10]. The prototype is intentionally more limited and easier to audit.

3.2 Why a rational multiplier is not enough

The usual presentation writes $G(n, k) = R(n, k)F(n, k)$ with a rational function R and a hypergeometric term F . Away from poles, a rational identity can show

$$\sum_i a_i(n)F(n+i, k) = G(n, k+1) - G(n, k).$$

At a support boundary, however, F may vanish while R has a pole. Declaring $RF = 0$ there because “the summand is zero” is not a valid argument. Cancelling a rational expression on an interior domain does not choose its extension at a singular endpoint.

For example, even the relation

$$\binom{n}{k-1} = \frac{k}{n-k+1} \binom{n}{k}$$

requires care at $k = n+1$. Its left side is 1, while the right presentation has a zero denominator multiplied by a zero binomial coefficient. The shifted binomial is the well-defined object. The rational presentation is only a tool for algebra on the interior.

The safe certificate therefore stores the flux as a sum of *shifted summands*, not as an unevaluated singular product.

3.3 The shifted-support ansatz

Extend the binomial coefficient to a signed lower index by

$$\binom{n}{k} = 0 \quad \text{when } k < 0 \text{ or } k > n,$$

with its ordinary value otherwise. For an order $r \geq 1$ recurrence, search for univariate polynomials $a_0, \dots, a_r \in \mathbb{Q}[n]$, with $a_r \neq 0$, and bivariate polynomials $P_0, \dots, P_{r-1} \in \mathbb{Q}[n, k]$. Define, for $k \geq 0$,

$$G(n, k) = \sum_{j=0}^{r-1} P_j(n, k) q^k \binom{n+j}{k-1}^p. \quad (5)$$

The desired summand identity is

$$\sum_{i=0}^r a_i(n) q^k \binom{n+i}{k}^p = G(n, k+1) - G(n, k), \quad 0 \leq k \leq n+r. \quad (6)$$

The two end fluxes are structurally zero:

$$G(n, 0) = 0, \quad G(n, n+r+1) = 0. \quad (7)$$

Indeed, the first uses the signed index -1 , and in the second $k-1 = n+r > n+j$ for every $j < r$. No limiting argument is involved.

The fixed summation family is part of the authoritative problem. A certificate contains only the coefficient polynomials. It cannot choose a different summand, change p or q , or assert its own support convention.

3.4 Interior verification by a polynomial identity

On $0 \leq k \leq n$, every factor of

$$D(n, k) = \prod_{h=1}^r (n + h - k)^p$$

is positive. Put

$$R_i(n, k) = \prod_{h=1}^i (n + h)^p \prod_{h=i+1}^r (n + h - k)^p, \quad (8)$$

$$T_j(n, k) = k^p \prod_{h=1}^j (n + h)^p \prod_{h=j+2}^r (n + h - k)^p. \quad (9)$$

Empty products are one. The binomial quotient identities give

$$R_i = D \frac{\binom{n+i}{k}^p}{\binom{n}{k}^p}, \quad T_j = D \frac{\binom{n+j}{k-1}^p}{\binom{n}{k}^p}.$$

The checker verifies the exact sparse-polynomial equality

$$\sum_{i=0}^r a_i R_i - \sum_{j=0}^{r-1} q P_j(n, k+1) R_j + \sum_{j=0}^{r-1} P_j(n, k) T_j = 0. \quad (10)$$

This implies (6) on the interior. Only nonzero binomial and D factors are used in the rational derivation. The factor q^k is multiplied back, *never divided out*. Therefore the proof also covers $q = 0$, with the ordinary convention $0^0 = 1$.

All arithmetic in the acceptance checker is addition and multiplication of sparse rational polynomials. Search-generated residual expressions are not accepted on assertion: the checker reconstructs the residual from the authoritative problem and the supplied coefficients, then requires its canonical dictionary to be empty.

3.5 Every exceptional boundary is a separate checked identity

The interior covers $k \leq n$, but (6) is needed through $n + r$. For each $t = 1, \dots, r$, set $k = n + t$. Define the polynomial

$$B_{j,t}(n) = \begin{cases} \binom{n+j}{j-t}, & j-t \geq 0, \\ 0, & j-t < 0. \end{cases}$$

When the lower index is nonnegative, it is fixed and the binomial is a polynomial in n with rational coefficients:

$$\binom{n+j}{d} = \frac{1}{d!} \prod_{h=0}^{d-1} (n + j - h).$$

The checker verifies, for every $t = 1, \dots, r$,

$$\begin{aligned} 0 = & \sum_{i=0}^r a_i(n) B_{i,t}(n)^p \\ & - q \sum_{j=0}^{r-1} P_j(n, n+t+1) B_{j,t}(n)^p \\ & + \sum_{j=0}^{r-1} P_j(n, n+t) B_{j,t-1}(n)^p. \end{aligned} \quad (11)$$

This is again a polynomial identity, now univariate. Multiplying it by q^{n+t} gives the exact boundary summand identity. There is no assumption $q \neq 0$. Complementing the binomial lower index is justified because $n \geq 0$ and the nonnegative fixed lower indices are no larger than the upper indices.

Theorem 3.1 (Boundary-safe telescoping). *If $a_r \neq 0$ and the polynomials satisfy (10) and all r identities (11), then*

$$\sum_{i=0}^r a_i(n) S_{p,q}(n+i) = 0 \quad (n \in \mathbb{N}). \quad (12)$$

Proof. The interior calculation and the r boundary calculations establish (6) at every integer k in $[0, n+r]$. Sum that identity over this whole interval. Each summand $\binom{n+i}{k}^p$ vanishes above $n+i$, so its sum is exactly $S_{p,q}(n+i)$, even though the different shifted sums have different natural support endpoints. The flux telescopes to $G(n, n+r+1) - G(n, 0) = 0$ by (7). This proves (12). $\square$

The theorem is a deliberate alternative to a general-purpose rational singularity solver. A restricted flux language makes the endpoints and support coverage cheap to prove. Broader summands should enter through additional proved support schemas, not by weakening this theorem’s side conditions.

3.6 How the search is implemented

For chosen bounds d_a, d_P , write

$$a_i(n) = \sum_{a=0}^{d_a} c_{i,a} n^a, \quad P_j(n, k) = \sum_{a+b \leq d_P} d_{j,a,b} n^a k^b.$$

Substitute these templates into (10) and every identity (11). Equating coefficients produces a homogeneous rational linear system. Its number of unknowns is

$$(r+1)(d_a+1) + r \binom{d_P+2}{2}. \quad (13)$$

The producer computes its exact nullspace using SymPy, selects a basis vector with nonzero highest recurrence coefficient, clears rational denominators, removes the common integer content, and chooses a positive leading coefficient for a_r . It then sends the certificate through the independent checker.

Including the boundary equations in the search is important for completeness within this ansatz. Computing only the interior nullspace and testing its individual basis vectors against the boundaries could miss a useful linear combination. The delivered implementation includes all those equations before nullspace calculation. If every nullspace basis vector has $a_r = 0$, so does every linear combination, and this particular order and degree request has no admissible solution. The worker reports `UNKNOWN_FIXED_ANSATZ`, not “no recurrence exists” or “the theorem is false”.

For the fixed finite linear system, the selection of a top-nonzero nullspace vector is complete. This does *not* imply completeness of the union of this particular flux language over all bounds, nor of creative telescoping for arbitrary expressions. The number of monomials and coefficient bit sizes can also make an exact nullspace calculation expensive. The quartic-power example below already takes most of the measured telescoping search time.

3.7 Recovered certificates, not just guessed recurrences

Binomial theorem. For $p = 1$, one certificate is

$$a_0 = -(1 + q), \quad a_1 = 1, \quad P_0 = -1.$$

Thus $S(n + 1) = (1 + q)S(n)$ and $S(0) = 1$. The flux is $-q^k \binom{n}{k-1}$. This is a simple sanity case, not evidence that Forge needs to rediscover an identity already available in Mathlib's binomial library [17]. A production router should prefer a direct library theorem when it matches.

Squared binomials. For $p = 2, q = 1$, the search returns

$$a_0 = -2(2n + 1), \quad a_1 = n + 1, \quad P_0 = 2k - 3n - 3. \quad (14)$$

The recurrence is $(n + 1)S(n + 1) = 2(2n + 1)S(n)$. Combined with the elementary factorial recurrence of $\binom{2n}{n}$ and the initial value 1, it proves

$$\sum_{k=0}^n \binom{n}{k}^2 = \binom{2n}{n}.$$

The artifact establishes the summation certificate, not an already integrated Lean proof of this closed form. At $n = 0$, the boundary cell $k = 1$ contributes exactly the term needed for the recurrence. Dropping it is not harmless simplification of a zero tail.

More generally, the following formula can be checked by the displayed polynomial identities for every rational q :

$$\begin{aligned} a_0 &= (q - 1)^2(n + 1), & a_1 &= -(q + 1)(2n + 3), & a_2 &= n + 2, \\ P_0 &= (q - 1)(n + 1), & P_1 &= 2k - 3n - 6. \end{aligned}$$

Substitution into (10) and (11) proves the parametric statement by polynomial expansion. The executed search fixes q before solving its rational system; it did not infer this as a universal parameterised theorem. A separate supplied SymPy script also expands all three residuals with symbolic q and obtains zero; this check is outside the main rational-certificate corpus. The corpus checks nine order-two rational instances, including 0, negative values, and nonintegral values, in addition to the order-one $q = 1$ instance.

Alternating squared binomials. For $q = -1$ the order-two recurrence becomes

$$(n + 2)S(n + 2) + 4(n + 1)S(n) = 0,$$

with flux polynomials

$$P_0 = -2(n + 1), \quad P_1 = 2k - 3n - 6.$$

The initial values are $S(0) = 1, S(1) = 0$. Consequently odd values vanish, and the even subsequence is $S(2m) = (-1)^m \binom{2m}{m}$. The stride-two nature of this example illustrates why a worker should retain the whole recurrence operator, including a zero middle coefficient, rather than assume every identity reduces to a first-order ratio.

Franel numbers. For $p = 3, q = 1$, the emitted operator is

$$(n + 2)^2 S(n + 2) - (7n^2 + 21n + 16)S(n + 1) - 8(n + 1)^2 S(n) = 0. \quad (15)$$

Its explicit safe flux uses

$$\begin{aligned} P_0(n, k) &= -4(n+1)^2, \\ P_1(n, k) &= -6k^2 + 15kn + 30k - 10n^2 - 40n - 40. \end{aligned}$$

This request has $r = 2, d_a = d_P = 2$, a 54×21 search matrix, nine boundary rows, and a one-dimensional nullspace. A smaller first-order request with $d_a = d_P = 1$ has nullity zero and is preserved as an unknown result. The initial values are 1, 2, and the leading coefficient $(n+2)^2$ has no natural zero. Thus this particular recurrence needs no additional singular seed values.

A larger, nonminimal certificate. For $p = 4, q = 1$, the chosen bounds $r = d_a = d_P = 3$ produce a valid order-three operator. Its last coefficient is $7(n+3)^3$; the full operator and three flux polynomials are stored in the JSON corpus. The 159×46 matrix has a two-dimensional nullspace. This is an acceptance of a correct recurrence, not a claim that order three is minimal. Requiring minimal order in the checker would add cost without strengthening the theorem established by a valid annihilator.

3.8 Lean-specific traps and the initial support library

The most important translation trap is the lower index $k - 1$. In Lean’s `Nat`, subtraction is truncated: $0 - 1 = 0$. Therefore translating $\binom{n}{k-1}$ with a natural lower index would make the supposedly zero left flux nonzero. The safe interface needs a signed lower index, for example an explicitly defined `chooseZ : Nat -> Int -> Rat`, zero for negative indices, with proved connections to `Nat.choose` on nonnegative inputs. Alternatively, pattern match on k before using natural subtraction. Either choice must be reflected in the denotation proof.

An initial Lean support library should prove: zero outside the binomial support; the interior shift-ratio identities with their inequalities; the complement symmetry needed at the top boundary; the fixed-lower-index polynomial product formula; finite-sum interchange; and the telescoping endpoint theorem. Mathlib’s `choose` and `finite-sum` libraries supply relevant building blocks, but the exact certificate theorem still has to be written and compiled. A generic `field_simp` invocation is not a substitute for the missing domain proofs.

The source reifier must also distinguish the finite range $\{0, \dots, n\}$ from $\{0, \dots, n - 1\}$. In a `Finset.range` encoding, (4) uses `n + 1`. The common support used in the proof has length $n + r + 1$. Off-by-one changes to either range alter the theorem.

The first support schema is intentionally finite. Infinite sums would require convergence and limiting boundary proofs. Products of factorials with several moving bounds require a richer support partition. Neither is admitted by the present checker; neither should be approximated by checking a large finite range and silently invoking the finite theorem.

4 Worker III: recurrence transport with singularity-aware seeds

4.1 Why two good recurrence proofs may not meet

Suppose one worker proves that a sequence f is annihilated by A , while a different derivation proves that g is annihilated by B . Matching their first several values does not automatically establish equality. One needs a common induction rule and an exact account of the indices where that rule cannot propagate equality.

Use the forward shift E , acting by $(Ef)(n) = f(n+1)$. A polynomial-coefficient operator is

$$A = \sum_{i=0}^r a_i(n)E^i, \quad (Af)(n) = \sum_{i=0}^r a_i(n)f(n+i).$$

The coefficients lie in $\mathbb{Q}[n]$, and all recurrence hypotheses in the prototype are intended for every natural n . The order is the largest index with a nonzero coefficient polynomial.

4.2 Noncommutative multiplication is part of the certificate

Because shifting changes a coefficient, operator multiplication obeys

$$(a_i(n)E^i)(b_j(n)E^j) = a_i(n)b_j(n+i)E^{i+j}. \quad (16)$$

In particular $En = (n+1)E$, not nE . The supplied checker expands this identity coefficientwise using exact polynomial shift. A dedicated unit test rejects commutative multiplication as a substitute.

A certificate of a common *left* multiple consists of three operators L, U, V such that

$$L = UA = VB. \quad (17)$$

The problem supplies A, B ; the certificate cannot redefine them. The checker requires all three polynomials-in- E to be canonical with nonzero final coefficients and verifies both products by (16). It does not claim L is a least common left multiple.

Proposition 4.1 (Annihilator transport). *If $Af = 0$ and $Bg = 0$ on $\mathbb{N}$, and (17) holds with polynomial coefficients and nonnegative shifts, then $Lf = Lg = 0$ on $\mathbb{N}$.*

Proof. For all n , $(UA)f(n) = U(Af)(n) = 0$, since every shifted index remains natural. The analogous calculation applies to $V(Bg)$. Identity (16) is exactly what makes composition of actions agree with operator multiplication. $\square$

The prototype searches for U, V of bounded order and coefficient degree, forms the linear equations for $UA - VB = 0$, and solves them exactly. Any nonzero solution yields an acceptable L . In a production system, `ore_algebra` can supply more sophisticated left-multiple and desingularisation algorithms; its output should still pass this simpler polynomial identity check [10, 11]. Rational-function operator coefficients would require a denominator policy and validity-domain ledger. The implemented format excludes them.

4.3 A concrete common multiple

Let

$$A = E - (n+1), \quad B = E - 2(n+1).$$

The search returns

$$U = E - 2(n+2), \quad V = E - (n+2),$$

and

$$L = E^2 - 3(n+2)E + 2(n+1)(n+2).$$

Expanding UA and VB with (16) verifies the result. Treating E as a commuting indeterminate would produce the wrong middle coefficient. This example is a general operator-composition test, not a claim that the nonzero factorial and doubled-factorial solutions of the two first-order operators are equal. Equality still needs the relevant initial-value proofs.

The certificate also permits a deliberate nonminimal common multiple. This helps separate discovery from acceptance: if a producer finds a larger L quickly, the checker can accept it without proving an optimality theorem. The cost is potentially more seed obligations and additional apparent singularities, which must be accounted for rather than cancelled informally.

4.4 The singularity problem

Write a common recurrence as

$$\sum_{i=0}^r p_i(n)f(n+i) = 0, \quad p_r \neq 0, \quad r \geq 1. \quad (18)$$

At an index n with $p_r(n) \neq 0$, the preceding r values determine $f(n+r)$. At a root of p_r , that argument fails. Merely knowing that p_r is a nonzero polynomial is insufficient: a polynomial can be nonzero as an expression and zero at a specific natural index.

Example 4.2 (A missed singularity invalidates prefix reasoning). Consider

$$(n-5)(f(n+1) - 2f(n)) = 0.$$

The zero sequence and

$$g(n) = \begin{cases} 0, & n < 6, \\ 2^{n-6}, & n \geq 6 \end{cases}$$

both satisfy it. They agree at indices 0 through 5 and disagree at index 6. Thus even six matching initial values do not justify equality for this first-order recurrence unless the singular step is handled. This countermodel is exercised by the unit tests.

4.5 A selective seed theorem

Let

$$\Sigma = \{s \in \mathbb{N} : p_r(s) = 0\}, \quad I = \{0, 1, \dots, r-1\} \cup \{s+r : s \in \Sigma\}.$$

Because p_r is a nonzero polynomial over a field, Σ is finite.

Theorem 4.3 (Singularity-aware recurrence equality). *Let $f, g : \mathbb{N} \rightarrow \mathbb{Q}$ both satisfy (18). If $f(i) = g(i)$ for every $i \in I$, then $f(n) = g(n)$ for every $n \in \mathbb{N}$.*

Proof. Set $h = f - g$. It satisfies the same homogeneous recurrence. Use strong induction on m . If $m < r$, then $h(m) = 0$ by the seed assumptions. Otherwise write $m = n + r$. If $p_r(n) = 0$, then $m \in I$ and the seed assumption again applies. If $p_r(n) \neq 0$, the recurrence at n reads

$$p_r(n)h(m) = -\sum_{i=0}^{r-1} p_i(n)h(n+i) = 0,$$

because $n+i < m$ for every $i < r$. Cancellation in the field gives $h(m) = 0$. $\square$

This set of seeds is sufficient, not asserted minimal. Additional recurrence relations can sometimes determine a value that this theorem requests separately. Its advantage over demanding every value through the largest singular index is that it asks only for the initial window and the exceptional pivots. Repeated roots do not cause duplicate seed obligations.

For the example above, the required set is $\{0, 6\}$. For a third-order recurrence with natural leading roots 0, 1, 5, it is $\{0, 1, 2, 3, 4, 8\}$. A seed at s instead of $s+r$ is an off-by-order error; it does not repair the missing pivot.

4.6 A complete finite cover of natural singularities

The checker must not trust a producer-supplied list of roots. Let

$$p_r(n) = a_d n^d + \cdots + a_0, \quad a_d \neq 0.$$

For $d > 0$, choose an integer

$$B \geq 1 + \max_{0 \leq i < d} |a_i/a_d|. \quad (19)$$

Every complex root has absolute value at most this bound. For completeness, if $|z| > 1 + C$ with $C = \max_{i < d} |a_i/a_d|$, then a zero would imply

$$|z|^d \leq C \sum_{i=0}^{d-1} |z|^i \leq \frac{C}{|z| - 1} |z|^d < |z|^d,$$

a contradiction. The constant-polynomial case has no roots and uses $B = 0$.

A singularity-cover certificate supplies B , the claimed natural roots, and the proposed seed set. The checker verifies (19), evaluates p_r exactly at every integer in $[0, B]$, compares the complete sorted root list, and recomputes I . It performs no floating-point root approximation and does not trust a solver's assertion that all roots were found.

This enumeration is intentionally simple and sometimes inefficient. Large coefficients can make B enormous. The prototype refuses bounds above 100 000, returning unknown from the producer. That is a resource policy, not a root-existence conclusion. A later positive-factor or Sturm cover could reduce work. Forge already proposes Sturm machinery, so reusing a proved version is preferable to declaring another root-counting implementation new.

Evidence status. The 47 executed singularity objects verify complete seed *plans* for their encoded operators. They do not contain proofs that two arbitrary sequences satisfy the recurrences, and they do not prove the seed equalities.

4.7 Apparent singularities introduced by transport

A second recorded example starts from $A = E - (n + 1)$ and $B = E - 2$. The producer returns a polynomial common multiple with leading coefficient $n - 1$:

$$L = (n - 1)E^2 + (-n^2 - 3n + 2)E + 2n(n + 1).$$

Its multipliers are

$$U = (n - 1)E - 2n, \quad V = (n - 1)E - n(n + 1).$$

Although both original operators are monic, this common multiple has a natural singularity at 1. The safe equality plan includes the additional seed at $1 + 2 = 3$. One may be able to choose another operator or exploit the originals to avoid that extra work. One may not simply discard the factor or omit the seed because the singularity “came from the algorithm”.

This observation gives the scheduler a concrete optimisation criterion: compare candidate common multiples by checked operator size, estimated verification cost, and the number and difficulty of their residual seed goals. It is not necessary to introduce a new global scheduling architecture to use that cost signal.

4.8 Connecting all three workers

For a unary rational matrix M , the rows $u, uM, \dots, uM^d$ are linearly dependent. Exact search can propose scalars c_i with $\sum_i c_i uM^i = 0$. Multiplication by $M^n v$ gives a constant-coefficient recurrence for the output sequence. This is another small identity certificate, and can also be obtained through a characteristic polynomial. That extraction is a proposed extension, not part of the recorded runner.

A hypergeometric sum can independently receive a telescoping recurrence. A claimed closed form, generating-function coefficient sequence, or library sequence may carry yet another annihilator. Their verified operators meet through (17); Theorem 4.3 supplies the final obligations. In the simplest squared-binomial example, both sides already share a first-order operator, so common-multiple search should be skipped.

The resulting proof plan is explicit:

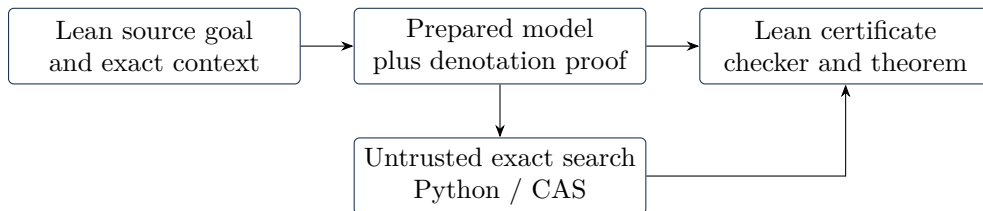
source correspondence $\longrightarrow$ two proved annihilation statements
 $\longrightarrow$ checked common operator $\longrightarrow$ complete singularity cover
 $\longrightarrow$ proved seed equalities $\longrightarrow \forall n, f(n) = g(n)$.

Each arrow has an independent obligation. No stage is allowed to stand in for one of the stages after it.

5 Concrete Lean integration, with no new global controller

5.1 Reuse Forge’s boundary; add worker-specific semantics

Forge already specifies a neutral intermediate representation, exact origin ownership, worker isolation, and checked-fact publication. Repeating those requirements at length would not improve the design [4, 7]. The new implementation work is the mathematical meaning of the three prepared problem types and the proofs connecting them to source expressions.



The denotation proof belongs to preparation, before the external answer exists. The external search supplies coefficients and indices, not executable Lean source. Its positive result has to satisfy the checker’s own reconstructed identities for the already prepared model. The final proof is transported back to the original goal. This instantiates Forge’s existing boundary rather than inventing a competing one.

5.2 The smallest useful word-fold reifier

Begin with equalities of rational-valued computations over the *same List Rat* input and fixed rational initial states. Recognise either an explicit state fold or a pair of folds that can be synchronised into a product state. The first implementation can restrict all state fields to rationals and all updates to affine expressions in those fields. Do not start with arbitrary recursive Lean functions.

A practical preparation algorithm is:

```

prepareWordGoal(goal):
  expose the equality and its exact quantified list binder
  recognize approved fold definitions and their traversal direction
  synchronize same-input folds using a proved product-fold lemma
  reify initial state, update, and output difference
  reject unsupported operations and unresolved coefficient parameters
  choose the output-degree monomial lift
  generate update and observation denotation proofs
  return fixed matrices, variable origins, and correspondence theorem

```

The expression reifier accepts rational literals, addition, subtraction, multiplication, and fixed natural powers, with degree bookkeeping. Division is initially accepted only when it is part of a rational literal with a proved nonzero numeral denominator. A multiplication of two state-dependent affine expressions fails the affine-update test, although it can be valid in the observer. A polynomial input variable can occur in update coefficients; its post-lift degree must be calculated, not guessed.

A fold-right computation and a fold-left computation are not synchronised by merely using the same list variable. Their relationship needs a proved traversal lemma. Likewise a mapped, reversed, filtered, or indexed input is not the original word unless a source-level transformation theorem justifies that view. The initial source subset should refuse these cases, then add them with explicit correspondence lemmas.

For a goal with arbitrary initial parameters, there are two principled future routes. One can introduce a finite polynomial coefficient family of initial rows and share a reachable-space closure across those rows. Or one can ask Forge’s existing generalisation planner to propose and prove a stronger state invariant. The current runner uses concrete initial states; it does not implement either extension as a source tactic.

5.3 The matrix soundness theorem is smaller than the search

In Lean, represent the rectangular certificate matrix using finite index types, for example `Matrix (Fin b) (Fin d) Rat`. Define the embedding

$$\text{embed}(x) = xB,$$

the full transition $s \mapsto sM_a$, and the small transition $x \mapsto xC_a$. The identity $BM_a = C_aB$ proves that the embedding commutes with transition. The identity $Bv = 0$ proves the observation vanishes on the embedded state. This is exactly the generic word-simulation argument.

The bundle includes `lean/WordSimulation.lean`, a proposed core Lean proof of that generic argument, with no matrix search hidden in it. It is marked *uncompiled*; the working environment did not provide Lean. It is not a reflected checker, a tactic, or an evidence receipt. Its intended use is to separate the straightforward induction lemma from the subsequently required matrix implementation.

For a first compiled implementation, construct a proof of each finite matrix identity with ordinary rational normalisation and instantiate the soundness lemma. For larger certificates, implement a reflected Boolean checker and prove its correctness once. The latter optimisation should come after a real small-certificate end-to-end path works. Exact elimination need never be verified inside Lean merely to check the returned matrix identities.

5.4 Summation preparation and residual obligations

A summation preparer must recognise the exact range, integer exponent p , rational weight q , cast structure, and binomial family. The first entry point should accept the literal form (4); equivalent-looking factorial quotients become eligible only through proved rewrite lemmas. It returns the family parameters and a theorem equating the source sum with its prepared semantics.

The checked telescope publishes a recurrence theorem as an intermediate fact. It should not pretend to close an arbitrary requested closed form. Forge can then use an available target recurrence, ask an appropriate library adapter for one, or leave that target-side recurrence as an explicit obligation. If the source goal already asks for the recurrence, publication closes that part directly.

The recurrence equality worker should generate a proof graph containing both annihilation proofs, the operator-product equality proof, the singularity cover, and every seed equality. A seed proof may come from `norm_num`, direct finite computation through the kernel, a library lemma, or another worker. If one seed remains unresolved, the result is a partial proof plan, not a positive theorem result. This is a direct use of Forge’s existing obligation graph, not a new graph data structure.

5.5 Proposed modules and concrete responsibilities

The following paths are proposed additions to Forge; they do not denote files already installed in the repository.

Module	Responsibility and acceptance invariant
<code>Forge/Summary/WordModel.lean</code>	Rational row-state semantics; word evaluation; finite alphabet identity.
<code>Forge/Summary/WordCertificate.lean</code>	Typed B, α, C data; dimension-safe identity checker; proved soundness.
<code>Forge/Summary/PolynomialInput.lean</code>	Coefficient-matrix semantics; interpolation theorem; actual-grid counterexample reconstruction.
<code>Forge/Reify/AffineFold.lean</code>	Exact approved fold grammar; monomial lift; source denotation proof; explicit refusals.
<code>Forge/Summary/BinomialSupport.lean</code>	Signed-lower-index binomial; support, ratio, complement, and endpoint lemmas.
<code>Forge/Summary/Telescopers.lean</code>	Polynomial identities and boundary partition; recurrence theorem for the fixed sum family.
<code>Forge/Summary/ShiftOperator.lean</code>	Polynomial Ore product and sequence action; left-multiple transport theorem.
<code>Forge/Summary/RecurrenceSeeds.lean</code>	Complete natural-singularity cover and strong-induction equality theorem.
<code>Forge/Worker/FiniteSummary.lean</code>	Bind prepared goals to solver requests; check results; publish proofs or residual obligations through existing interfaces.

Keep the first three mathematical modules independent of `grind` internals. Their output is an ordinary theorem. A controller can feed that theorem into a fresh local `grind` call through Forge’s already proposed conservative bridge. No changes to congruence closure or the Boolean search loop are necessary for these first capabilities.

5.6 Library map: use existing mathematics and search engines

Library	Concrete use	Status in this study
Lean core / Mathlib	Finite indices, lists, matrices, rational arithmetic, polynomial evaluation, finite sums, choose identities, and proof tactics	API/module documentation inspected; no local Lean build.
SymPy 1.14.0	Exact polynomial expansion and rational nullspaces for bounded telescoping and operator search	Executed by the delivered producer.
Python standard library	Rational matrix and sparse-polynomial certificate checking with <code>Fraction</code> ; canonical JSON decoding	Executed in isolated <code>python -S</code> replay.
<code>python-flint</code>	Optional fast dense rational matrices, inversion, characteristic polynomials, and elimination backend	Official <code>fmpq_mat</code> documentation consulted; adapter not implemented.
Sage <code>ore_algebra</code>	Optional LCLM, operator arithmetic, and desingularisation search, returning polynomial certificates	Official capabilities inspected; not installed or benchmarked here.
HolonomicFunctions	Optional broader multivariate creative-telescoping search and Ore computations	Official package description inspected; no Wolfram execution in this study.

The referenced Mathlib matrix-to-linear-map and span modules contain the infrastructure relevant to the simulation theorem. The choose-sum module already includes direct binomial identities, so those should be tried before expensive rediscovery [15, 16, 17]. SymPy’s matrix API supplies the nullspace operation used here; its role is proposal generation, not proof acceptance [13]. The rational matrix type in `python-flint` is an appropriate candidate for larger exact searches, but no speedup has been measured [14].

Forge’s Lean-status ledger pins Lean 4.34.0 and a Mathlib revision, while also warning that other proposed dependencies are not automatically build-compatible with that pin [8]. The Lean release itself is recorded as 4.34.0 [18]. The new workers should use the repository’s resolved project environment rather than independently chasing current versions of all libraries. The delivered Python requirement is separately pinned to the actual executed version. No fabricated Lake manifest is supplied.

6 A specific Leant/Djex extension: quantified equivalence of eligible folds

6.1 The gap to target

Leant already generates source-typed candidates and checks supplied behavioral assertions on the exact candidate. Its documented assertion strategy uses positive and negative decision attempts and bounded simplification; finite observations establish only the stated observations. A universal assertion can remain inconclusive when it needs a mathematical argument beyond those procedures [19]. Djex’s source-typed evidence graph preserves exact source ownership and the relationship between a candidate and its checked graph [20]. Forge already incorporates the generic ownership lessons from those projects [7]. The addition proposed here is a particular semantic consumer: an exact affine-fold equivalence check.

Given two eligible rational-valued list folds at the same source type, translate each to a linear word model with correspondence proofs. Form their block comparison model and invoke Worker I. A valid closure certificate gives equality for every list, not just the finite list test set. A separating word is a useful new behavioral discriminator, subject to the source correspondence discipline.

6.2 An operational candidate pipeline

```

for each exact, source-typed candidate occurrence:
  try the proved affine-fold interpretation
  if unsupported: preserve the ordinary behavioral-check route
  otherwise compare to the approved specification model
  if a matrix closure certificate is returned:
    kernel-check it and transport to the exact candidate proposition
  if a separating word is returned:
    reconstruct the actual source input and verify its observed failure
  preserve candidate ownership, original budgets, and residual goals

```

The specification model must not be inferred from the candidate’s own type alone. A type such as `List Rat -> Rat` admits many different behaviors. The user assertion or an approved specification theorem determines the intended observation. Likewise, a candidate’s exact occurrence must not borrow a graph or proof receipt from another candidate with the same rendering.

The source bridge should begin with ordinary list folds, not arbitrary polymorphic Church encodings. A theorem about encoded finite lists is not automatically a theorem about all inhabitants of an abstract Church type. Extending the bridge to those inputs needs the appropriate source semantics or parametricity argument; it must not be assumed from a type-directed search result. The original Leant integer-indexing and simultaneous-universe search queries are outside the tested fragment and are not claimed solved here.

6.3 Semantic deduplication that does not discard evidence

A proved functional equality can justify sharing future *extensional* behavioral checks between eligible candidates. Maintain an equivalence class whose edges are actual equalities, while retaining each candidate’s own origin and source typing evidence. A chosen representative does not acquire the others’ metadata. Transport an assertion along an equality only when the assertion has the correct source type and Lean accepts that transport.

It is unsafe to use a matrix-model equality as a universal deduplication key for arbitrary typed programs. The translation may cover only one observer, one input domain, or one source specialisation. Even a full extensional equality is not definitional equality and does not justify rewriting unrelated provider or universe metadata. The useful optimization is narrower: avoid repeating a behavioral proof that can actually be transported through a checked equality.

6.4 Counterexamples improve search without becoming oracles of truth

For an exact model, a separating word can be returned directly with a proof of the relevant source inequality once the source bridge is in place. For an over-approximate model, rerun the candidate on the reconstructed input before using the observation to reject it. An infeasible abstract transition sequence must not eliminate a source-correct candidate.

This provides a concrete feedback loop: previously found counterexample words are cheap tests for later candidates; surviving candidates still need the universal matrix certificate. Passing the accumulated examples never replaces that proof. The new work is the finite-dimensional universal

argument, not the already proposed general counterexample-guided synthesis architecture.

7 Executed evidence and what it measures

7.1 The authoritative run

The delivered code was run under Python 3.13.5 and SymPy 1.14.0 on the recorded Linux container. Its deterministic random seed is 681437. The authoritative outputs are in `results/run-4/`. The numbers below belong to that one corpus and are not pooled with Forge’s nine earlier prototypes.

Accepted object	Count	Invalidating mutations rejected
Rational matrix closure	54	54
Concrete separating word	115	115
Shifted-support telescope	23	23
Common-left-multiple identity	6	6
Complete singularity/seed plan	47	47
Total	245	245

Table 4: Objects checked, not 245 Lean theorems. The 47 plans still require their residual sequence and seed proofs.

The whole recorded corpus, including searches, symbolic lifting, numerical cross-checks, and mutations, completed in approximately 5.558 seconds in this run. Separate replay of all stored certificates took approximately 0.339 seconds with Python site packages disabled and SymPy absent from the replay process. These are different workloads, not a controlled search-versus-check speedup claim. They are also not a prediction of Lean kernel checking time. The unrounded observations and exact environment string are retained in JSON.

The replay command is

```
python -S prototype/replay.py results/run-4/certificates.json
```

Its output explicitly records `site_packages_enabled: false` and `sympy_loaded: false`. The checker does use the same small exact-arithmetic representation module as the producer’s serializer. It is search-independent, not an independently authored or formally verified implementation of arithmetic.

7.2 Composition of the word corpus

One hundred random rational models have dimension 1 through 5, twenty of each dimension, with two letter matrices. Their search outcomes were compared to exhaustive evaluation of every word of length at most $d - 1$. There were 1140 such word evaluations, and all one hundred model-level comparisons agreed. This uses the proven finite distinguishing bound; it is not arbitrary finite sampling presented as universal truth.

Forty positive comparison models were constructed by invertible rational coordinate changes of random systems of dimension 2 through 6, eight per dimension. Their block comparison dimensions are twice the original dimensions. These exercise nontrivial rational coefficients and certificates of equality between different representations. They are generated positives with a known mathematical construction, not a representative sample of user theorems.

Twenty-three delayed-separation models of dimension 2 through 24 return first nonzero output at word length exactly $d - 1$. They show why checking a fixed small prefix is insufficient even in this

elementary linear fragment. The remaining six models are the Cassini lift, four polynomial-input Horner instances, and the degree-two input-grid counterexample. The combined word corpus therefore has 169 entries: 54 closures and 115 actual separating words.

Representative model	Lift dimension	Emitted basis rank	Expansions
Cassini, either one or two advances	10	8	16
Horner relation, each of four bases	4	3	6
Input-grid trap $x(x - 1)$	2	1 before separation	3
Delayed separation, dimension 24	24	23 before separation	23

Table 5: Expansions count one stored-row/letter multiplication, not Lean tactic steps.

The polynomial lift checks its own symbolic substitution identities before search. This is useful producer-side validation, but its results do not test an independent Lean reifier. That distinction remains visible in the capability ledger.

7.3 Summation and recurrence evidence

Twenty-five prescribed telescoping requests were attempted. Twenty-three returned certificates and two had zero nullspace at their declared first-order bounds. The successful family consists of eleven $p = 1$ weights $q = -5, \dots, 5$; ten $p = 2$ instances, including negative, zero and fractional weights; one Franel instance; and one fourth-power instance. Both bounded misses are stored with their order, degree bounds, matrix dimensions, and nullity.

For every accepted recurrence, direct exact summation checked $n = 0, \dots, 20$, for 483 additional recurrence checks. The runner also checks every summand cell from $k = 0$ through $n + r$ on those inputs. These finite cross-checks are supplementary error detection. The universal acceptance is the polynomial identity and boundary certificate, not agreement on the first twenty-one values.

Request	r	d_a, d_P	Matrix	Boundary rows	Nullity
Binomial, $q = 1$	1	0, 0	4×3	1	1
Squared binomial, $q = 1$	1	1, 1	11×7	2	1
Alternating squared binomial	2	1, 1	27×12	6	1
Franel	2	2, 2	54×21	9	1
Fourth power	3	3, 3	159×46	24	2
Franel, undersized request	1	1, 1	17×7	2	0
Fourth power, undersized request	1	1, 1	22×7	2	0

Table 6: Actual search systems. A zero nullspace is local to these prescribed bounds.

The six common-multiple tests include constant operators, equal operators, a zero constant coefficient, and nonconstant polynomial coefficients. Each checks both exact left products. The singularity corpus consists of one plan for every accepted telescoper and twenty-four additional operators: eight prescribed natural root sets at each of orders one, two, and three. The root sets include zero, positive separated roots, and multiple exceptional pivots.

7.4 Invalidating changes and independent unit tests

Each stored entry receives one deliberately invalidating mutation. For a matrix closure, the authoritative initial row changes while its certified coordinates do not. For a separating word, the claimed nonzero output is replaced by zero. For a telescoper, one is added to its last operator coefficient, so the required interior residual changes by the nonzero polynomial R_r . For an Ore identity, one is added to its last common-operator coefficient while the multipliers are unchanged.

For a seed plan, the first required seed is removed. All 245 of these specified mutations are rejected.

These are not random byte flips, nor a claim that every syntactic modification should be rejected. Scaling a valid entire telescope by a nonzero rational, for instance, can produce another valid certificate and should be accepted. The mutation suite tests known-invalid changes to the mathematical subject or identity.

An additional 41 unit tests cover noncanonical rationals, floats and Boolean values masquerading as integers, repeated monomials and JSON keys, degree bounds, wrong output or transition matrices, missing letter actions, wrong initial coordinates, a zero initial row, empty separating words, non-affine updates, insufficient observation degree, $q = 0$, changed weights and fluxes, zero operators, the Ore commutation trap, omitted singular roots or reset seeds, understated root bounds, and the explicit recurrence countermodel from Section 4. Their full verbose output is retained.

7.5 Failed runs and repairs are retained

The first complete run reached Ore search and failed because the zero polynomial had become a Python integer lacking a SymPy substitution method. The original traceback is preserved. Coercing zero into a SymPy expression repairs that path, and a dedicated zero-coefficient regression test passes.

An intermediate run passed with interior-only search followed by boundary checking. The search was then strengthened to include boundary rows before nullspace calculation, avoiding the basis-combination completeness issue. The first run of that revision exceeded the 45-second execution limit before writing results. Inspection found a Python division of an empty product that could introduce a floating coefficient. The delivered code uses an explicit SymPy rational multiplier and checks that every matrix entry is rational before solving. The exact timeout location was not profiled, so the artifact does not claim that this diagnosis alone proves its runtime cause. The corrected final revision completes and supplies the article’s measurements.

Historical results are labelled separately from the authoritative run. A past passing certificate file is not evidence that a changed search implementation was exercised. Conversely, fixing a producer does not justify deleting its failed-run record.

7.6 What these experiments cannot establish

No Python experiment here measures Lean elaboration time, kernel checking cost, realistic theorem retrieval, or interaction with `grind`. There is no head-to-head comparison with `grind`, `Aesop`, `nlinarith`, `LeanHammer`, a CAS-backed tactic, or Forge’s other workers. The generated corpus is designed to exercise algorithms and failure modes, not to estimate a user-development success rate. No extrapolated percentage improvement is reported.

The decoders impose bounds on dimensions, degrees, rational bit lengths and collection sizes, but all intermediate arithmetic has not been hardened against resource exhaustion. A full hostile-input security audit and broader fuzzing remain future work. The processes do not evaluate certificate text as code, but that alone does not make an implementation resistant to denial of service.

Most importantly, the complete source-to-Lean-proof path is untested. A correct matrix model of the wrong fold would prove the wrong statement. A correct recurrence plan with unproved seeds remains only a plan. These limitations should be acceptance gates, not footnotes erased by a successful demonstration.

8 Implementation priorities and acceptance gates

8.1 First increment: one real matrix proof path

The first pull request should not introduce every proposed worker. It should compile a word semantics module, a matrix simulation theorem, a checked finite certificate format, and one approved fold reifier. The existing Forge controller can invoke this through ordinary theorem construction.

A convincing first acceptance contains a theorem about *every* list, such as Example 2.4, proved from a generated certificate for the exact original goal. It also contains a wrong-update control that is genuinely rejected and a separating input that evaluates to an actual source inequality. A successful proof for a prewritten auxiliary matrix problem is useful but does not close the source-reifier acceptance gate.

The first release can admit only rational source state and inputs. This avoids simultaneously implementing ring-homomorphism transports, natural-number cast normalisation, and guarded branches. A deliberately small implemented fragment is more valuable than a large interface that silently abstracts unsupported operations.

8.2 Second increment: the signed-support telescoping theorem

Formalise the signed-lower-index binomial semantics and prove Theorem 3.1. Then reconstruct the squared-binomial and Franel certificates from the exact emitted polynomials. Acceptance must include $n = 0$, $q = 0$, $k = 0$, and $k = n + r$ cases, not just a generic interior calculation. A test that changes natural subtraction into the signed predecessor semantics must fail unless an appropriate case lemma justifies it.

Only after that theorem compiles should an external CAS adapter admit additional hypergeometric summands. Each new support schema needs its own interior domain, exceptional set, and endpoint proof. A universal “simplify the rational certificate” adapter is not an acceptable replacement for those obligations.

8.3 Third increment: operator action and selective seeds

Prove that polynomial Ore multiplication agrees with sequence-operator composition, then prove Theorem 4.3. Start with already matched operators; add common-multiple search after the simpler equality pipeline is working. The singularity cover should be checked independently of the producer’s root-finding method.

Acceptance must include an operator with a natural singularity and a countermodel agreeing on the ordinary initial window. The positive proof must explicitly use the additional pivot seed. A certificate that omits it must not close the goal. A common multiple that introduces an apparent singularity must remain subject to the same rule.

8.4 Fourth increment: the Leant consumer

Integrate only the approved fold fragment through the exact candidate and source-graph interface. The acceptance result should be a universally quantified assertion about an actual emitted candidate, with its own original context and type, not a theorem about a hand-supplied reference implementation. Keep the ordinary assertion checker as a fallback for unsupported candidates.

A semantic-equivalence cache becomes useful only after equality transport is implemented and measured on real candidate streams. Begin with a small counterexample-word bank; it offers

cheap early rejections while leaving positive acceptance to the universal proof. Do not alter existing candidate budgets or count a larger observation window as the success of this worker.

8.5 A fair future tactic evaluation

There are two different claims to test. *Coverage* asks whether the new worker proves original goals that the existing configured system does not prove within its declared budget. *Efficiency* asks what the additional worker costs, including preparation, search, proof reconstruction, and kernel checking. Both require actual Lean runs.

A useful comparison uses four fixed configurations: the existing local baseline; Forge without the new workers; Forge with the word worker; and Forge with all successfully integrated summary workers. Keep imports, available lemmas, heartbeats, wall-clock limits, and source goals identical. Record a separate configuration when a generic enabling lemma is supplied. Do not turn a failed original goal into an accepted easier auxiliary statement.

The benchmark should contain renamed and structurally varied fold examples, changing rational coefficients, higher observation degrees inside the admitted lift, moving-support sums, parameter edge cases, and singular recurrences. It must also include unsupported and false inputs, where the right result is refusal or a checked counterexample rather than a fabricated proof. Keep synthetic unit families separate from held-out repository theorems.

A useful engineering metric is the distribution of *final proof-checking* cost against certificate dimension, polynomial term count, and coefficient bit length. Another is the fraction of candidate models rejected by the reifier before search. Those measurements will indicate whether to prioritise a faster external solver, a reflected checker, or a broader source fragment. The present Python timings cannot choose among those Lean bottlenecks.

9 Conclusion

The most valuable expansion of Forge is not an unbounded promise to “do more induction”. It is a collection of precise finite objects that make particular infinite proofs routine. A transition-closed row space proves an identity for all words. A polynomial coefficient decomposition extends that proof to an infinite input alphabet. A shifted-support flux removes a hypergeometric sum without inventing values at singular products. A common Ore operator and a complete singularity cover turn independent sequence proofs into a finite, explicit list of remaining equality obligations.

These additions fit Forge’s existing architecture. They reuse ordinary Lean proofs as the publication interface, retain the separation between search and acceptance, and exploit established mathematical software rather than reimplementing every solver. They also give Leant and Djex a sharply defined new behavioral capability: proving equality of eligible folds for every input, not merely observing agreement on a finite test set.

The experiments establish that the proposed discovery and checking mechanisms are executable, that a separate exact replay does not need the symbolic search package, and that the specified false certificates are rejected. They do not establish a working Lean tactic. The next decisive step is therefore concrete: compile the finite matrix soundness theorem, connect it to one exact source fold, and obtain a kernel-checked proof for that original universally quantified goal. The other workers can then grow behind equally explicit acceptance gates.

A Artifact protocol and reproduction

A.1 Files and commands

The ZIP archive contains this article and its complete multi-file LaTeX source, the Python producer and checker modules, the unit tests, raw run records, and the uncompiled Lean semantic lemma. Run from the extracted bundle root:

```
# Install only the producer dependency in an appropriate Python environment.
python -m pip install -r prototype/requirements.txt

# Recheck the supplied corpus; no third-party Python packages are loaded.
python -S prototype/replay.py results/run-4/certificates.json

# Exercise the standalone regression suite.
cd prototype
python -m unittest discover -s tests -v
cd ..

# Recreate the full corpus in a new or empty directory.
python prototype/run_experiments.py --out results/reproduction

# Supplemental universal-q polynomial check (uses SymPy).
python prototype/check_parametric_square.py

# Rebuild the article with a normal TeX installation.
cd article
latexmk -pdf -interaction=nonstopmode -halt-on-error \
  forge_finite_summaries.tex
```

The producer deliberately refuses a nonempty output directory. This avoids mixing a changed run's measurements with old files. The historical run records are described in `results/RUN_HISTORY.md`. The supplemental parameterised square-sum check establishes zero symbolic residuals in n, k, q ; it is not counted among the 245 rational-problem entries and is not a Lean proof.

A.2 Canonical rational and polynomial encoding

A rational is encoded as `[numerator, denominator]`, with integer components, a strictly positive denominator, and greatest common divisor one. The zero rational is therefore exactly `[0, 1]`. Floats, Booleans, noncanonical fractions, and negative or zero denominators are rejected.

A polynomial is a sorted list of terms `[nExponent, kExponent, numerator, denominator]`. Zero terms and duplicate monomials are forbidden. An empty list denotes the zero polynomial. A univariate operator coefficient must have zero k exponent. An operator is an array of these polynomials, indexed by shift power; its last coefficient must be nonzero.

The checker fixes a maximum rational bit length of 4096, total polynomial degree 128, and 4096 terms per polynomial. Word models have dimension at most 64 and at most 32 explicitly supplied transition matrices. The telescoper decoder further restricts power to 1 through 8 and order to 1 through 6. General operator decoding permits order at most 16. The singularity cover has a bound cap of 100 000. These input limits are research safeguards, not a comprehensive bound on all intermediate resource use.

The file replay loader rejects duplicate JSON keys and nonfinite numeric constants and imposes a 32 MiB file-size cap. It does not execute strings as Python or Lean source. Structural shape

checks precede the mathematical identities. This is preferable to a producer-controlled “residual is zero” field: no such field is accepted.

A.3 Authoritative problem versus certificate

Every replay bundle has exactly the top-level fields `checker`, `problem`, and `certificate`. A word problem supplies `kind`, `dimension`, `initial`, `output`, and `transitions`. Its closure certificate supplies `kind`, `basis`, `initial_coordinates`, and `actions`. A word counterexample instead supplies its literal word and a nonzero claimed value, which the checker recomputes against the authoritative matrices.

For example, with $u = (1, 1)$, $v = (1, -1)^T$, and letter matrices $2I, 3I$, a one-row certificate has $B = (1, 1)$, $\alpha = (1)$, and actions $(2), (3)$. The universal equality follows even though the two state coordinates may grow without bound. Changing only the initial row to $(2, 1)$ invalidates the certificate and is caught by $u = \alpha B$ before any word search is performed.

A telescoping problem supplies only its fixed family tag, power, and rational weight. The certificate supplies operator and flux arrays. A common-multiple problem supplies both original operators; its certificate supplies the proposed common operator and left multipliers. A recurrence-plan problem supplies its operator; the certificate supplies a bound, complete natural root list, and seed list. Extra object fields are rejected rather than ignored.

This format makes one kind of subject-substitution error observable: changing the problem while retaining the old certificate generally fails a recomputed identity. In production, the problem also needs the source denotation proof and exact context binding. JSON shape alone does not provide those properties.

A.4 Checker map

Entry point	Mathematical work performed
<code>check.linear</code>	Decode dimensions and rationals; verify $u = \alpha B$, $Bv = 0$, and $BM_a = C_a B$ for all supplied letters.
<code>check.counterexample</code>	Re-evaluate the concrete word in the original model and compare its nonzero rational output.
<code>check.telescopier</code>	Reconstruct (8)–(9); verify the interior identity and every exceptional boundary polynomial.
<code>check.ore_identity</code>	Apply the shifted polynomial product rule twice and compare with the proposed common left multiple.
<code>check.singularity_plan</code>	Verify a Cauchy cover, enumerate all natural roots in it, and recompute the required pivot seeds.

The producer modules are separate: `linear_search.py`, `telescoping_search.py`, `ore_search.py`, and `lift.py`. Neither `check.py` nor `exact.py` imports SymPy or any search routine. The latter is a deliberately small shared arithmetic and encoding module; sharing it is disclosed because it is not the same as a fully independent second implementation.

B Scope ledger for future claims

Before describing a future result as “Forge proved it”, require the exact source goal, the accepted source fragment, the produced certificate, the reconstructed Lean proof, the kernel outcome, and

the actual axiom inventory. Before describing a future result as “faster”, require a controlled baseline with the same goal, imports, available lemmas, and budgets.

The current archive establishes exact model-level search and replay, mathematical soundness arguments, a documented implementation route, and bounded experimental evidence. It does not establish the source-to-kernel path. The capability ledger in `docs/STATUS.md` is intended to survive integration: each row has a separate acceptance condition and cannot be closed by a nearby easier result.

References

- [1] V. Reshetnikov, *Forge repository README*, revision 674521027d968d59f7b83220ed52304a85cb55e2, inspected 15 September 2026. Pinned source.
- [2] V. Reshetnikov, *Forge capability status*, same revision. `docs/STATUS.md`.
- [3] *Forge merged article: structural search*, same revision. `05-structural.tex`.
- [4] *Forge merged article: Lean integration*, same revision. `08-integration.tex`.
- [5] *Forge merged article: provenance*, same revision. `B-provenance.tex`.
- [6] *Forge merged article: conclusion*, same revision. `11-conclusion.tex`.
- [7] *Ideas from Leant and Djex*, Forge, same revision. Pinned transfer guide.
- [8] *Forge Lean status*, same revision. `docs/LEAN-STATUS.md`.
- [9] S. Kiefer, *Notes on Equivalence and Minimization of Weighted Automata*, 2020. arXiv:2009.01217. <https://arxiv.org/abs/2009.01217>.
- [10] M. Kauers, M. Jaroschek, and F. Johansson, *ore_algebra*, official RISC software description, inspected 15 September 2026. https://risc.jku.at/sw/ore_algebra/.
- [11] M. Kauers, M. Jaroschek, and F. Johansson, *Ore Polynomials in Sage*, 2013. arXiv:1306.4263. <https://arxiv.org/abs/1306.4263>.
- [12] C. Koutschan, *HolonomicFunctions*, official RISC software description, inspected 15 September 2026. <https://risc.jku.at/sw/holonomicfunctions/>.
- [13] SymPy contributors, *Matrices (linear algebra)*, SymPy 1.14.0 documentation, inspected 15 September 2026. <https://docs.sympy.org/latest/modules/matrices/matrices.html>.
- [14] python-flint contributors, *fmpq_mat—matrices over rational numbers*, official documentation, inspected 15 September 2026. https://python-flint.readthedocs.io/en/latest/fmpq_mat.html.
- [15] Mathlib contributors, *Mathlib.LinearAlgebra.Matrix.ToLin*, generated documentation, inspected 15 September 2026. https://leanprover-community.github.io/mathlib4_docs/Mathlib/LinearAlgebra/Matrix/ToLin.html.
- [16] Mathlib contributors, *Mathlib.LinearAlgebra.Span.Basic*, generated documentation, inspected 15 September 2026. https://leanprover-community.github.io/mathlib4_docs/Mathlib/LinearAlgebra/Span/Basic.html.
- [17] Mathlib contributors, *Mathlib.Data.Nat.Choose.Sum*, generated documentation, inspected 15 September 2026. https://leanprover-community.github.io/mathlib4_docs/Mathlib/Data/Nat/Choose/Sum.html.
- [18] Lean contributors, *Lean 4.34.0 release*, published 14 September 2026. <https://github.com/leanprover/lean4/releases/tag/v4.34.0>.
- [19] V. Reshetnikov, *Behavioral assertions during synthesis*, Leant. Main inspected 15 September 2026; resolved Leant main revision 6bf05ad78c467989e68290f2d08bbbed40802d485. Pinned guide.
- [20] V. Reshetnikov, *Djinn source-typed evidence graph*, Djex, main retrieved 15 September 2026. This guide was not assigned an immutable HEAD pin during the study. <https://github.com/VladimirReshetnikov/Djex/blob/main/docs/source-typed-evidence-graph.md>.